LAGOS STATE UNIVERSITY

CSC 392I · PROJECT REPORT

LASU Internship Management Portal

A browser-based system for internship placement applications, lecturer review and digital SIWES logbook tracking

Submitted in partial fulfilment of the academic requirements for CSC 392I.

Student name: _____

Matriculation number: _____

Department: _____

Project supervisor / lecturer: _____

Faculty / School: _____

Academic session: _____

Prepared from the supplied internship-management project source code and Supabase schema. The student and supervision fields are intentionally left blank for completion.

REPORT GUIDE

Executive Summary & Contents

This report describes a web portal that brings student internship placement submissions, lecturer decisions and weekly SIWES logbooks into one online workflow.

The inspected project uses HTML, CSS and browser JavaScript for its interface and application logic, with Supabase Authentication, PostgreSQL, private object storage and row-level security (RLS) for online accounts and records. Students can register, submit company and placement details, attach supporting files, check a decision and, after acceptance, maintain weekly log entries. Lecturers can review applications, record outcomes and feedback, approve logbooks, filter records and export application data.

> **Evidence and scope.** The description is based on the supplied repository files. It is an academic project report, not a claim of official LASU ownership, approval or production deployment. Seven VS Code–style figures are original explanatory illustrations based on the source; they are not photographs or captured screenshots of a running editor.

Table of contents

Chapter One — Introduction 3

- Overview of CSC 392I and project context 3
- Purpose, objectives and organisation of the report 3

Chapter Two — LASU Internship Management Project 4

- Project overview, problem statement and scope 4
- Core system functionality and user workflow 5
- System methodology and architecture 6
- Technology stack and data model 7
- Implementation illustrations (Figures 1–7) 8–11
- Proposed system enhancements, scope and limitations 11

Chapter Three — Conclusion 12

Chapter Four — References 13

Course context on the cover is supplied by the requester. Confirm the course's required format and fill in the student, supervisor and session details before submission.

CHAPTER ONE

Introduction

1.1 Background and overview of CSC 392I

CSC 392I is identified by the requester as the course context for this report at Lagos State University (LASU). The supplied project provides a practical software-engineering case study: requirements are translated into a browser interface, application workflows, a relational data model, authentication, file storage and access policies. No official course outline or marking rubric was provided; therefore, this chapter describes the project in that academic context without attributing unverified course outcomes to the University.

Industrial training administration involves more than recording a student's host organisation. Placement information must be submitted, reviewed, communicated and retained; weekly activities may also need to be documented. When these activities are handled through separate forms, email threads and spreadsheets, staff may have to reconcile information manually and students may not know the current state of a review.

1.2 Purpose of the report

The purpose is to explain the LASU Internship Management Portal: the administrative problem it addresses, its user roles, the main capabilities visible in the implementation, the technologies and data structures used, the development approach, and practical improvements that could make the system more secure, maintainable and accessible.

1.3 Report objectives

- Describe the portal's purpose, users, principal workflow and boundaries.
- Relate implemented screens and actions to the JavaScript and database design.
- Explain the Supabase-backed authentication, persistence, file handling and access controls.
- Identify realistic enhancements and an appropriate verification plan.

1.4 Purpose and organisation of the project

The system aims to provide a shared online record for internship applications and lecturer review, with a student-facing progress view and a weekly logbook that is available after placement acceptance. Chapter Two discusses the project and its implementation; Chapter Three presents the conclusion; Chapter Four lists project and technical references.

> **Terminology.** "Internship" and "SIWES" are used as they appear in the portal's interface. Local departmental requirements, approval rules and programme duration should be confirmed with the relevant LASU coordinator.

CHAPTER TWO

LASU Internship Management Project

2.1 Project overview

The LASU Internship Management Portal is a small web application composed of a landing page and an internship portal. Students can create an account using their matriculation number and contact details, provide company and placement information, submit required documents, monitor review status and enter weekly work activities. Lecturer accounts see a separate review dashboard for placement decisions and logbook approvals.

Application records are stored in Supabase's `applications` table. User profiles live in `profiles`, and weekly submissions are recorded in `logbook_entries`. File bytes are uploaded to a non-public Supabase Storage bucket; the corresponding file paths are included in application or logbook payloads. The browser requests short-lived signed links when a stored asset must be displayed.

2.2 Problem statement

A fragmented or paper-heavy placement process can make it difficult to identify which students have submitted the required paperwork, locate supporting documents, track pending decisions, return feedback and determine whether weekly training reports have been reviewed. Separate records can also introduce duplicate entry and make progress reporting time-consuming.

The project addresses this coordination problem with a central student submission and lecturer review workflow. A single record joins each application to its student account; status, reviewer feedback and logbook information are visible through role-specific screens. Online storage helps make records available across sessions and devices when the Supabase service is configured and reachable.

2.3 Project objectives

Student objectives

- Register and sign in with a matric number.
- Submit placement details and supporting documents.
- See application status and lecturer feedback.
- Record weekly activities after acceptance.

Lecturer objectives

- Review and filter placement submissions.
- Accept or deny an application and provide feedback.
- Inspect and approve logbook entries.
- Export application records as CSV.

2.4 Intended scope

The implemented scope covers student registration and login, placement submission and revision, document upload and retrieval, review status, duration progress, weekly logbook entries, lecturer review actions, CSV export and a printable clearance slip. It does not implement employer accounts, placement matching, payroll, automated academic grading or a dedicated mobile application.

Core System Functionality

2.5 Roles and principal workflow



2.6 Functional components

Component	Implemented behaviour	Main project evidence
Student account	Matric-number sign-in; sign-up details and a six-digit email code; student profile associated with an authenticated user.	`database.js`, `scripts.js`
Placement application	Captures student, department, school, supervisor, company, phone number, dates, notes and two required documents; supports saving an updated application.	`internship_management.html`, `database.js`
Status and progress	Displays Pending, Accepted or Denied status, lecturer feedback and date-based completion percentage.	`scripts.js`
Weekly logbook	Provides week, date range, tasks, skills and optional image fields; logbook entry form is locked until the application is Accepted.	`internship_management.html`, `scripts.js`
Lecturer dashboard	Summarises applications by status; offers search/filtering, decision actions, feedback, logbook review and CSV export.	`scripts.js`
File and clearance tools	Stores private supporting files, creates signed links for access and presents a printable clearance slip.	`database.js`, `internship_management.html`

2.7 User-facing state changes

After sign-in, the portal loads the user's profile and records. Student navigation opens application status, application form, weekly logbook or clearance slip; lecturer navigation opens the review dashboard and record export. A newly submitted logbook updates the application's review state to "Awaiting lecturer approval." Lecturer approval updates both the application and its entries.

> **Review gate.** The interface and submission handler both check that a placement is Accepted before a student can submit a weekly log. Database row-level policies should remain the authoritative access boundary and must be tested against direct API requests as well as the UI.

System Methodology

2.8 Development method

The repository reflects an iterative, feature-oriented web development approach. User-facing HTML defines the screens and forms; CSS supplies layout and themes; JavaScript coordinates validation, navigation and actions; and a Supabase SQL schema supplies persistent records, triggers, policies and storage rules. This structure makes a short demonstration flow possible while keeping the main data model and backend access rules explicit.

1. **Identify roles and requirements:** distinguish student submission and lecturer review responsibilities.
2. **Design the workflow and data:** define profiles, applications, weekly entries, their relationships and file paths.
3. **Implement interface and actions:** connect forms and dashboard controls to authentication, database and storage operations.
4. **Validate end-to-end behaviour:** test signup/login, ownership, file access, review decisions, logbook gating and persistence.
5. **Deploy and maintain:** configure Supabase and EmailJS, publish static web assets and apply reviewed schema changes.

2.9 Logical architecture



The browser is the presentation and orchestration layer; the Supabase project provides cloud authentication, database access and object storage. EmailJS is called from the browser to send the registration code. This is a client-integrated design, not a server-rendered application or a custom application server.

2.10 Persistence and access strategy

The database schema links a profile to an authenticated user ID, permits one application per student, and stores weekly entries separately. Row-level security policies restrict student queries to the signed-in owner and permit lecturer reads based on the lecturer profile role. A database trigger creates a profile when an Auth user is registered; another trigger updates application logbook review state when an entry is inserted. Storage policies keep the bucket private and scope upload/read/delete operations by user folder and role.

> **Deployment prerequisite.** The provided setup instructions include a clean-start SQL schema that drops and recreates tables, policies and the storage bucket. Applying it to a populated project can destroy records or files; production deployments should use reviewed, incremental migrations and verified backups.

Technology Stack & Data Model

2.11 Technology stack

Technology	Role in the project
HTML5	Landing page, login/signup views, application and logbook forms, dashboards and dialogs.
CSS3	Responsive layout, dashboard styling, light/dark theme tokens and printable clearance-slip presentation.
JavaScript (browser)	Form handling, input checks, authentication calls, dashboard state, date calculations, upload orchestration and CSV export.
Supabase JavaScript client	Connects the browser to Supabase Auth, PostgreSQL data APIs and Supabase Storage.
Supabase Auth + PostgreSQL	User sessions and role-linked profiles; application and logbook persistence; triggers, constraints, indexes and RLS.
Supabase Storage	Private `portal-files` bucket for acceptance forms, application letters and logbook images.
EmailJS	Browser-side delivery of the registration verification code and status notification messages.
Visual Studio Code	Suitable editor for browsing the source, changing interface logic, inspecting SQL and running local checks.

2.12 Core records and relationships

Record	Key data	Relationship / rule
`profiles`	Auth user ID, matric, names, email, role	One profile per Auth user; role is student or lecturer.
`applications`	Student ID, matric, JSON payload, status, feedback, logbook review fields, timestamps	One application per student; status constrained to Pending, Accepted or Denied.
`logbook_entries`	Student ID, matric, JSON payload, approval flag and creation time	Multiple weekly entries may belong to a student.
Storage objects	Uploaded file content under a user-scoped path	Private bucket; database payload holds references, not public links.

The JSONB payload permits the application and weekly activity fields to evolve without adding a separate SQL column for every form input. Important workflow state remains in typed columns such as status, feedback and approval flags.

> **Account design note.** The current onboarding associates a matric number with a Supabase account and uses a surname-derived password strategy. Before operational deployment, use a unique, user-chosen strong password (or an institution-approved identity provider), avoid shared predictable credentials, and review email-code verification on the server side.

Implementation Walkthrough

The following original VS Code–style editor illustrations explain the supplied source structure. They are schematic figures, not literal application screenshots.

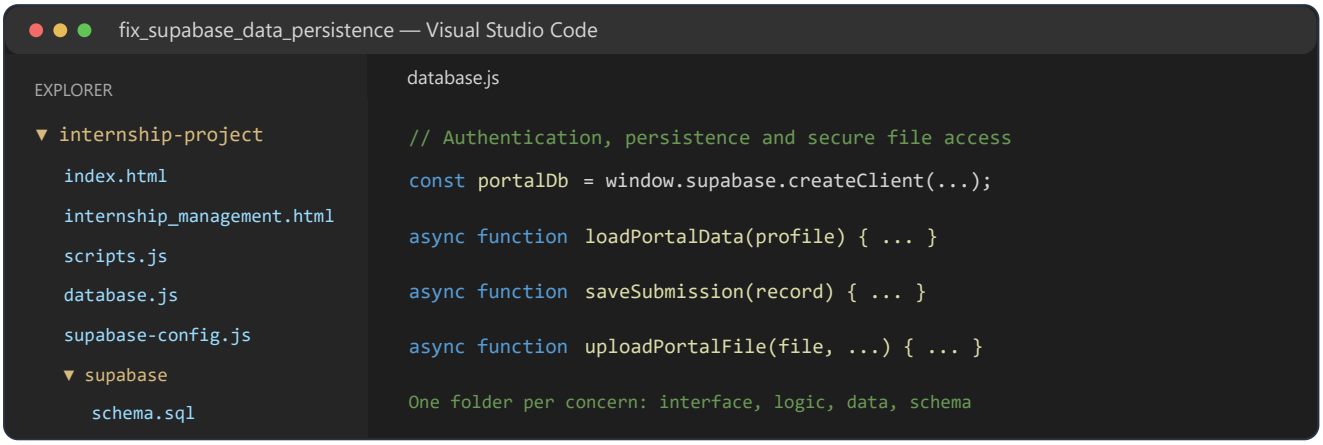


Figure 1. Project Explorer and separation of the application's main responsibilities.

Open the repository folder in VS Code. The two HTML files define the landing page and portal screens; `scripts.js` handles user interactions; `database.js` centralises Supabase operations; `supabase/schema.sql` defines the database and policies.



Figure 2. Portal form markup groups placement details and supporting-document inputs.

The real application form includes identity, department, supervisor, company, dates, two supporting files and optional notes. The browser's `required` attributes provide basic form checks; the submit handler uploads selected files and saves the application.

Implementation Walkthrough (continued)

database.js · student account flow

```
async function createStudentAccount (pending) {
    const result = await portalDb.auth.signUp({ ... });
    if (result.error) throw ...;
    const profile = await getProfile(result.data.user.id);
    if (!profile || profile.role !== 'student') throw ...;
return { session, profile };
```

Figure 3. Authenticated account creation is followed by a profile lookup and role check.

The signup path first collects student identity and contact information, sends a verification code and then creates a Supabase Auth account after code entry. A profile check confirms that the user is configured as a student before the application proceeds.

supabase/schema.sql · core tables

```
create table public.profiles (
    id uuid primary key references auth.users(id),
    matric text unique,
    role text check (role in ('student', 'lecturer'))
);
-- applications: one per student; logbook_entries: weekly records
```

Figure 4. SQL schema excerpt highlights identity, matric uniqueness and role constraints.

Three relational tables support the workflow: profiles, one-per-student applications, and multiple weekly logbook entries. Typed review columns and constraints protect important status values; JSONB payloads hold the form-specific details.

Implementation Walkthrough (continued)

database.js · private document storage

```
const path = `${user.id}/${matric}/${category}/${name}`;
await portalDb.storage.from('portal-files').upload(path, file);
// Persist the storage path with the application payload
await portalDb.storage.from('portal-files')
      .createSignedUrl(path, 3600);
Private object → scoped, temporary viewing URL
```

Figure 5. Uploaded files use user-scoped paths and temporary signed URLs for access.

The project uploads application documents and optional weekly images into a private bucket. Its payload stores file paths; hydration requests signed URLs valid for a limited time rather than making the entire bucket public. Storage policies restrict access to the owner and, for reads, the lecturer role.

supabase/schema.sql · row-level security

```
create policy "Students and lecturers can read applications"
on public.applications for select to authenticated
using (student_id = (select auth.uid())
    or (select public.is_lecturer()));
-- Policies enforce record-level access in the database
Authenticated identity + owner / lecturer role
```

Figure 6. Row-level security scopes reads to a student's own record or an authorised lecturer.

The SQL schema enables RLS and defines policies for profiles, applications, logbooks and storage objects. A lecturer-role function checks the signed-in profile. The UI's role-specific navigation is not a substitute for these server-enforced policies.

Enhancements, Scope & Validation

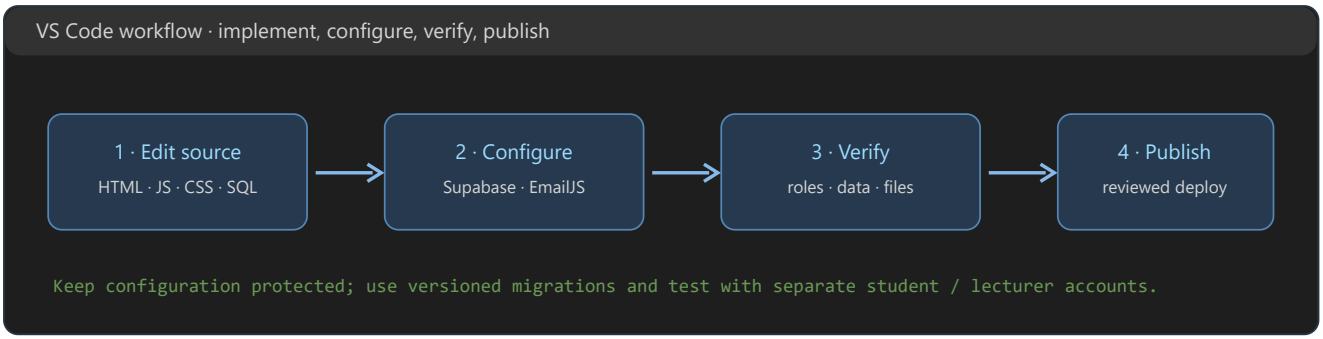


Figure 7. Suggested VS Code-centred development, configuration, validation and publishing cycle.

This process diagram is a recommended workflow for developing the supplied portal. It is not evidence that an automated test suite or production deployment currently exists.

2.13 Proposed system enhancements

1. **Strengthen account security.** Replace surname-derived passwords with strong, user-chosen credentials or institutional single sign-on; add password reset and multifactor options as appropriate.
2. **Move verification to a trusted service.** Verify email codes server-side, enforce attempt limits and expiry, and protect the delivery service from client-side abuse.
3. **Formalise authorisation and auditing.** Add least-privilege lecturer administration, database-level tests for RLS, reviewer identity and timestamps for decisions, and an audit history.
4. **Validate and protect uploads.** Enforce file type/size limits, malware scanning where available, safe filenames, and clear retention rules; test private signed-link expiry.
5. **Improve reliability and usability.** Add accessible labels and keyboard checks, clearer loading/error states, responsive usability tests, pagination and searchable CSV export.
6. **Adopt safe delivery practices.** Replace clean-slate schema resets with versioned migrations, back up data before changes, and use staging and production environments.

2.14 Scope limits and verification plan

The project is a lightweight static web application and depends on configured third-party services and network access. The report does not verify a live deployment, LASU integration, real student data, or institution-specific approval rules. Before release, test valid and invalid signup, duplicate matric values, each role's data visibility, application resubmission, file access as owner/non-owner, accepted-only logbook entry, lecturer review, persistence after logout and CSV content. Test database policies directly with student and lecturer test accounts, not only through the browser interface.

CHAPTER THREE

Conclusion

The LASU Internship Management Portal demonstrates how a digital workflow can connect student placement submissions, lecturer review and weekly training documentation in one browser-based system.

The implementation provides distinct student and lecturer views, a structured placement application, private file storage, review feedback, status and duration indicators, and a weekly logbook gated by placement acceptance. Supabase Auth associates accounts with profiles, while PostgreSQL tables retain applications and logbook entries. Row-level security and storage policies are intended to enforce ownership and lecturer access independently of the visible interface.

For a CSC 392I project report, the system is a useful practical example of front-end development, cloud database integration, authentication, data modelling and role-aware workflows. Its separation between HTML, CSS, browser logic and SQL schema makes those responsibilities identifiable and provides a foundation for extension.

Before operational use, the account and verification approach should be strengthened, upload validation and access rules should be tested, and the destructive clean-start schema should be replaced with a controlled migration process. A role-based test plan, dependable backups and user feedback would further improve confidence and maintainability.

> **Overall assessment.** The project meets the outline of a functional internship-management prototype. Its greatest value is the end-to-end workflow; its most important next step is production-grade verification of identity, authorization, file handling, operational data protection and deployment configuration.

Closing statement

With these improvements and validation against LASU's actual departmental procedures, the portal could provide a clearer, more consistent way to coordinate placement applications and SIWES progress records. Final acceptance for university use would require review and approval by the appropriate University authority.

CHAPTER FOUR

References

The following project files were inspected to describe the implemented features. External references provide the official technical context for the tools used. Online sources accessed October 2026.

1. **Internship Management Portal project source.** *index.html*; *internship_management.html*; *landing.css*; *styles.css*; *scripts.js*; *database.js*; *supabase-config.js*; *supabase/schema.sql*; and *supabase/SETUP.md*. Supplied project repository files, inspected October 2026.
2. **Supabase.** "JavaScript: Introduction." Supabase Documentation.
   https://supabase.com/docs/reference/javascript/introduction
3. **Supabase.** "Row Level Security." Supabase Documentation.
   https://supabase.com/docs/guides/database/postgres/row-level-security
4. **Supabase.** "Storage." Supabase Documentation.
   https://supabase.com/docs/guides/storage
5. **Supabase.** "Password-based Authentication." Supabase Documentation.
   https://supabase.com/docs/guides/auth/passwords
6. **MDN Web Docs.** "HTML forms." Mozilla.
   https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms
7. **Microsoft.** "Visual Studio Code Documentation."
   https://code.visualstudio.com/docs
8. **EmailJS.** "EmailJS Documentation."
   https://www.emailjs.com/docs/
9. **Lagos State University.** Official website.
   https://lasu.edu.ng/

Abbreviations

CSC	Computer Science (course prefix as supplied by the requester)	**LASU**	Lagos State University
SIWES	Student Industrial Work Experience Scheme	**RLS**	Row-level security
OTP	One-time password / verification code	**CSV**	Comma-separated values

> **Submission check.** Add your name, matriculation number, department, supervisor and academic session on the cover. Verify any department-specific CSC 392I formatting, citation style and project naming requirements before submission.